Raw Notes by Jolly Gupta

Came to know about Context7 via a Data Sense LinkedIn post and explored it on 16 April 2026.

Small steps initially matter the most. I move forward with curiosity and courage. In a world where versions constantly change, staying updated is essential—because even LLMs are limited by outdated knowledge.

Context7 Plugin in Claude: Why It Matters for AI Engineers

[Context7 - Up-to-date documentation for LLMs and AI code editors]

Context7 is how I get latest knowledge before coding

In tools like Claude, **one major limitation of LLMs is outdated knowledge**. Models are trained on past data, so they may not know the latest APIs, libraries, or updates.

This is where **Context7 plugin** becomes powerful.

What is Context7 Plugin?

Context7 is a documentation retrieval plugin that allows Claude to:

- Fetch **latest official docs**
- Pull **real-time API references**
- Inject them as context before generating answers

Think of it as:

"Live documentation layer added to Claude"

How It Works (Simple Flow)

1. User asks a question
   → "How to use latest OpenAI API?"
2. Context7 plugin
   → fetches latest documentation
3. Claude
   → uses that fresh context to generate answer

Why It's Important

1. Solves Outdated Knowledge Problem

LLMs have a cutoff date
Context7 → brings **up-to-date info**

2. Improves Code Accuracy

Without Context7:
❌ Deprecated functions
❌ Wrong syntax

With Context7:
✅ Latest methods
✅ Correct usage

3. Powerful for Developers

For AI engineers working with:

- APIs
- SDKs
- frameworks

Context7 ensures:
👉 You are always aligned with **current documentation**

Relation with RAG

Context7 is basically a **real-world implementation of** Retrieval-Augmented Generation

Difference:

- RAG → you fetch your own data
- Context7 → fetches **official docs automatically**

Real Insight

Without Context7:

AI = knowledgeable but outdated

With Context7:

AI = knowledgeable + **current + reliable**

Context7 actually works in BOTH places — they're two separate integrations:

1. Claude.ai chat (web/app) — as a plugin/connector You connect it directly on claude.ai (Tools menu → Context7 → Connect). Once connected, you just type use context7 in any chat message. No installation needed. This is what I showed in the guide above.

2. Code editors (Cursor, VS Code, etc.) — as an MCP server This requires the CLI setup and runs locally alongside your editor. Claude Code (the terminal tool) also uses this path.

CLI = Command Line Interface

It's a way to interact with your computer or tools by **typing commands instead of clicking buttons (GUI)**.

On your simplified flow — one correction needed:

The command npx ctx7 setup is **not the real command**. That package doesn't exist. The actual commands from official sources are:

Complete steps from scratch — Claude Code setup:

Press: Windows key + R Type: cmd Hit: Enter

Step 1 — Download & Install Node.js 18+

For Beginners

Go to: https://nodejs.org

Download: LTS version (18+ or 20+ recommended)

Run the installer → click Next → Next → Install

For Developers:

winget install OpenJS.NodeJS.LTS

Step 2 — Verify installation

bash

node -v # Should show v18.x.x or higher→ Node is installed, you're ready for next step.

npm -v # Should show a version number too

```
Node.js  →  runs JavaScript on your computer
            (not just in browser)

npm      →  Node Package Manager
            downloads & installs tools/libraries
            like Context7, Claude Code etc.
```

Step 3 — Install Claude Code

npm install -g @anthropic-ai/claude-code

Step 4 — Verify Claude Code

claude --version

Step 5 — Add Context7 MCP

claude mcp add context7 -- npx -y @upstash/context7-mcp@latest

Context7 works via:

- **MCP (Model Context Protocol)**
- not a simple npm package like “ctx7”

Step 6 — Restart Claude Code

exit

claude

Step 7 — Verify Context7 is connected

claude mcp list

Should show "context7" in the list

Step 8 — Test it

"How do I create a middleware in Next.js? use context7"

If Context7 fetches live docs before answering → working perfectly.

Common issues:

- node -v not found → restart terminal after installing Node
- claude not found → restart terminal after npm install -g
- Rate limited → get free API key at context7.com/dashboard

Why Context7 matters especially for RAG:

Library	Changes often?	Context7 helps?
LangChain	✅ Very fast	✅ Critical
LlamaIndex	✅ Very fast	✅ Critical
Pinecone SDK	✅ Yes	✅ Yes
Chroma	✅ Yes	✅ Yes

LangChain and LlamaIndex change **very frequently** — their APIs break between versions constantly. Context7 is especially valuable here because Claude's training data on these libraries goes outdated quickly.

What Context7 covers:

```
✅ LangChain          → latest versions, API changes
✅ LlamaIndex         → latest query engines, nodes
✅ Pinecone SDK       → latest upsert, query methods
✅ Chroma             → latest collection methods
✅ Weaviate           → latest client methods
✅ Qdrant             → latest vector operations
✅ OpenAI SDK         → latest embeddings API
✅ HuggingFace Hub    → latest model loading
✅ Supabase           → latest vector store methods
✅ MongoDB Atlas      → latest vector search
✅ pgvector           → latest Postgres setup
```

Context7 enhances RAG by adding a live retrieval layer that fetches up-to-date external data, improving accuracy, freshness, and reducing hallucinations.

Context7 → get latest docs/snippets

↓

Claude.ai / CLI → generate correct code (using those docs)

↓

Google Colab → run + test code

↓

(optional) enhance with live retrieval / improvements

✅ **Step 0: Give Prompt**

👉 Output = **latest, correct code**

✅ **Step 1: Copy Generated Code**

👉 From Claude / CLI

✅ **Step 2: Paste in Google Colab**

✅ **Step 3: Install Dependencies**

pip install -U langchain langchain-community langchain-openai chromadb

✅ **Step 4: Run Pipeline**

👉 Load → Split → Embed → Store → Retrieve → LLM

✅ **Step 5: (Optional) Add Context7-style Live Retrieval**

```
docs_db = vectorstore.similarity_search(query)
docs_live = fetch_live_docs(query)
final_docs = docs_db + docs_live
```

Your Final Mental Model

Step 0 → Get latest code using Context7 prompt (teacher gives correct notes 📋)
Step 1–4 → Build & run RAG (you study & write exam ✍)
Step 5 → Enhance with live data (you use internet for extra info 🌐)

Prompt in claude.ai

Method 1: Let the LLM Handle It (Best for you now)

Use latest LangChain documentation (Context7 style).

Build a RAG pipeline.

IMPORTANT:

- Use latest imports (langchain_community, langchain_openai)

- Avoid deprecated code

- If anything is outdated, automatically replace it

Also explain what changed from older versions. this is prompt for claud.ai but what for CLI?

Prompt in CLI:

Method 1: Prompt

Same Prompt Works in CLI Too

Method 2: One-Line Command (Pro Way)

claude "Use latest LangChain documentation (Context7 style). Build a RAG pipeline using latest imports and avoid deprecated code."

Complete Prompt:

Use the latest official documentation (Context7 style) to build a production-ready RAG pipeline.

Requirements:

Framework:

- Use latest LangChain (v0.2+ or newer)

- Also show equivalent pipeline using LlamaIndex (latest version)

Data Ingestion:

- Use modern document loaders (web, PDF, text)

- Explain when to use each loader

Chunking:

- Use RecursiveCharacterTextSplitter (latest)
- Explain chunk_size, overlap, and best practices

Embeddings:

- Use latest OpenAI embeddings (langchain_openai)
- Also mention HuggingFace embeddings as alternative
- Explain when to use which

Vector Database:

- Use Chroma (local)
- Also explain Pinecone / Weaviate / Qdrant (when to use each)
- Use latest APIs only

Retrieval:

- Use retriever (similarity search)
- Also implement advanced retrieval:
  - MMR (Max Marginal Relevance)
  - Top-k tuning

LLM:

- Use ChatOpenAI (latest)
- Show how context is injected into prompt

Pipeline:

- Build complete flow:

  Load → Split → Embed → Store → Retrieve → Generate

Code Requirements:

- Fully runnable in Google Colab
- Include pip install commands
- Use latest imports:

- langchain_community
- langchain_openai
- langchain_core
- Avoid deprecated code completely

Advanced (IMPORTANT):
- Add Context7-style live retrieval (optional layer)
- Show how to combine:
  vector DB results + live docs

Output:
- Clean, modular Python code
- Step-by-step explanation
- Highlight what changed from older LangChain versions

Bonus:
- Mention common errors and how to fix them

Make the code production-ready and beginner-friendly.

RAG-related prompts with Context7 — examples:

Basic RAG setup:

How do I build a RAG pipeline using LangChain? use context7

After using Context7:

1. Copy generated code
2. Paste in Colab
3. Install dependencies
4. Run pipeline
5. (Optional) Add live retrieval for better results

Note: same steps with other individual prompts

Vector database specific:

How do I store embeddings in Pinecone for RAG? use context7

How do I use Chroma as a vector store in RAG? use context7

How do I set up Weaviate for document retrieval? use context7

Embedding models:

How do I generate embeddings using OpenAI in a RAG app? use context7

How do I use HuggingFace sentence transformers for RAG? use context7

Chunking & document loading:

How do I split and chunk documents in LangChain for RAG? use context7

How do I load PDFs into a RAG pipeline using LlamaIndex? use context7

LlamaIndex specific:

How do I build a RAG query engine using LlamaIndex? use context7

How do I set up a vector index in LlamaIndex v0.10? use context7

Full pipeline questions:

Build me a RAG pipeline using LangChain + Pinecone + OpenAI embeddings. use context7

How Many Engineers Use Context7? (Reality)

1. GitHub Popularity (Strong Signal)

- Context7 MCP repo has ~**25,000+ stars**

👉 In developer world:

☆ 1K = small tool ☆ 10K+ = widely adopted ☆ 25K+ = **very popular**

2. Ecosystem Adoption

Context7 is:

- used in AI tools like Cursor, Claude Code, VS Code integrations
- integrated into developer workflows (IDE + AI coding tools)

👉 Meaning:

Thousands of engineers are actively using it

3. Scale of Documentation Coverage

- Supports **30,000+ libraries**
- Covers many frameworks + languages

👉 This only happens when:

- many developers rely on it

Realistic Estimate (Honest Answer)

No official count, but based on data:

> 💻 **Thousands to tens of thousands of engineers**
>
> 📈 Rapidly growing (especially in AI/GenAI space)